

Software Engineering

- Chapter 2

Unified Modeling Language(UML)

- Software Processes
 - Specification , Development, Validation, Evolution
- Software Process Models
-

Objectives

- To introduce software process
- To describe outline process models for requirements engineering, software development, testing and evolution
- To describe a number of different process models and when they may be used
- To introduce CASE technology to support software process activities

The software process

- Coherent sets of activities for **specifying, designing, implementing** and **testing** software systems (..set of **ordered** tasks) OR
- A structured set **of activities required to develop a software system**
 - **Specification** – the **functionality** of the **software** and **constraints** on its operation must be **defined** .
 - **Development** – the software to meet the specification must be **produced**
 - **Validation** – the software must be **validated** to ensure that it does what the customer wants
 - **Evolution** – the software must evolve to meet **changing** customer needs
- Activities vary depending on the organization and the type of system being developed.

Why we need software process

- Common understanding of the activities, resources and constraints involved in software development.
- Allow us to **examine**, **understand**, **control** and **improve** the activities that comprise the process.
- Captures our **experience** and **pass** them along to others
- Creating processes helps
 - Find **inconsistencies**,
 - **Redundancies**; and
 - **Omissions**

Software process characteristics

- Understandability
 - „ Is the process **defined** and **understandable**?
- „ Visibility
 - „ Is the process progress externally **visible**?
- „ Supportability
 - „ Can the process be supported by **CASE tools**?
- „ Acceptability
 - „ Is the process acceptable to those **involved** in it?
- Reliability
 - „ Are process **errors** discovered before they result in **product** errors?

- „ Robustness(STRONG AND HEALTHY)
 - „ Can the process continue in spite of **unexpected problems**?
- Maintainability
 - „ Can the process evolve to meet changing **organizational** needs?
- „ Rapidity
 - „ How fast can the system be **produced**?

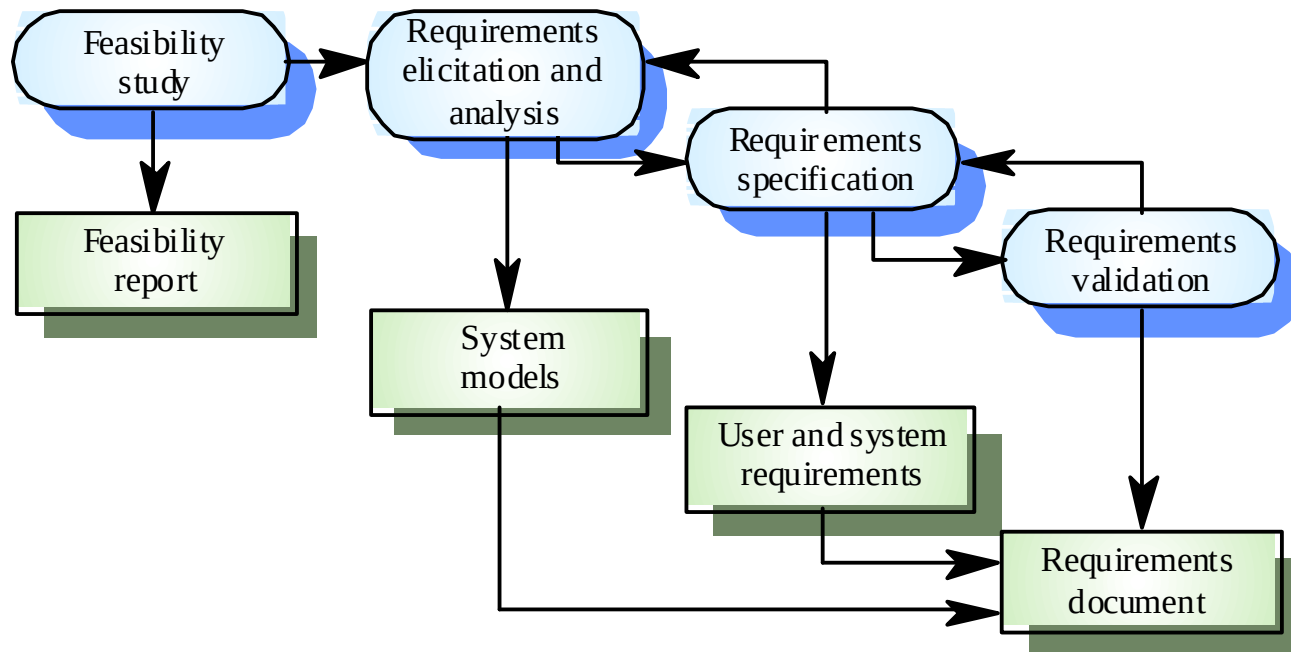
Process activities

- The four basic **process activities** of **specification, development, validation** and **evolution** are organized differently in different development processes.
- In the **waterfall** model, they are organized in **sequence**, whereas in **incremental** development they are **inter-leaved**.

Software specification

- The process of establishing what **services** are **required** and the constraints on the system's **operation** and **development**.
- Requirements engineering process
 - **Feasibility** study
 - Is it **technically** and **financially** feasible to build the system?
 - Requirements **elicitation** and **analysis**
 - What do the system stakeholders **require** or **expect** from the system?
 - Requirements **specification**
 - Defining the requirements in **detail**
 - Requirements **validation**
 - **Checking** the **validity** of the requirements

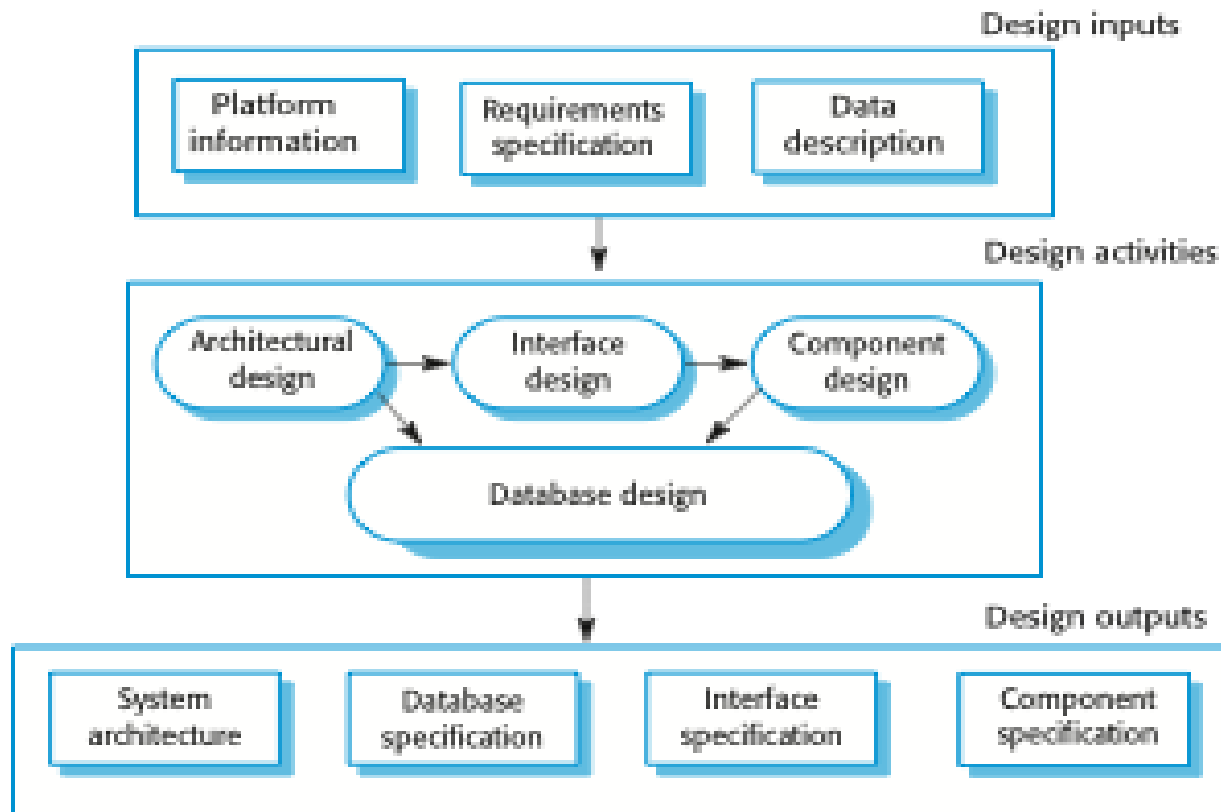
Requirements engineering process



Software design and implementation

- The process of converting the system specification into an executable system.
- Software design
 - Design a software structure that realises the specification;
- Implementation
 - Translate this structure into an executable program;
- The activities of design and implementation are closely related and may be inter-leaved.

A general model of the design process

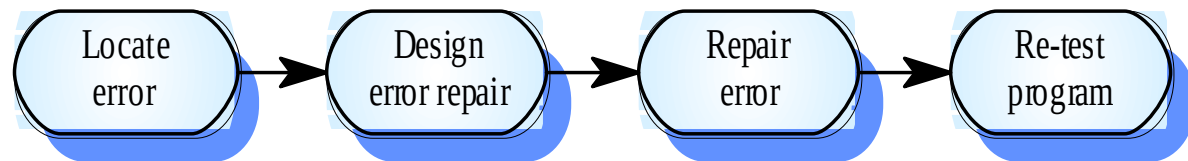


Design activities

- *Architectural design*, where you identify the overall structure of the system, the principal components (sometimes called **sub-systems or modules**), their relationships and how they are distributed.
- *Interface design*, where you define the **interfaces** between system components.
- *Component design*, where you take each system component and design how it will operate.
- *Database design*, where you design the system data structures and how these are to be represented in a database.

Coding - Programming and debugging

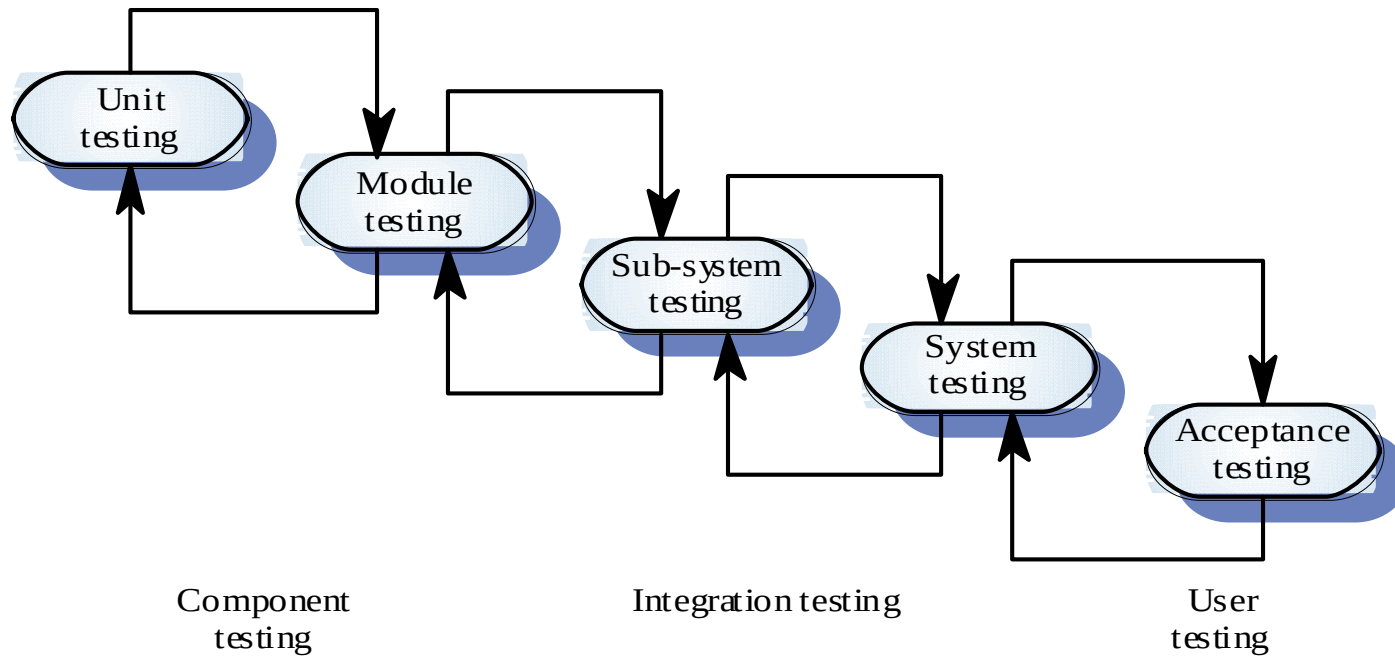
- Translating a design into a **program** and **removing** errors from that program
- **Programming is a personal activity** - there is **no generic programming process**
- **Programmers** carry out some program **testing** to discover **faults in the program** and remove these **faults** in the debugging process
- **The debugging process:**



Software validation

- **Verification** and **validation** (V & V) is intended to show that a system **conforms** to its **specification** and meets the requirements of the system **customer**.
- Involves **checking** and **review** processes and **system testing**.
- **System testing** involves executing the system with test cases that are derived from the **specification** of the real data to be processed by the system.
- **Testing** is the most commonly used **V & V** activity.

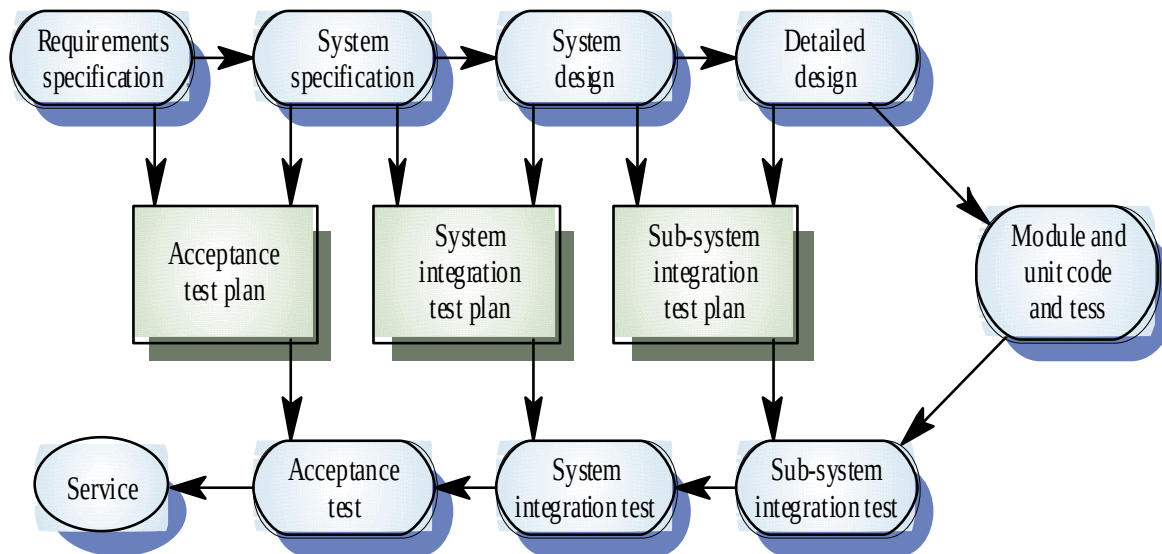
The testing process



Testing stages

- Unit testing
 - Individual components are tested
- Module testing
 - **Related** collections of **dependent** components are tested
- Sub-system testing
 - Modules are **integrated** into sub-systems and tested. The focus here should be on **interface** testing
- System testing
 - Testing of the system as a **whole**. **Testing of emergent** properties
- Acceptance testing
 - Testing with **customer** data to check that it is **acceptable**

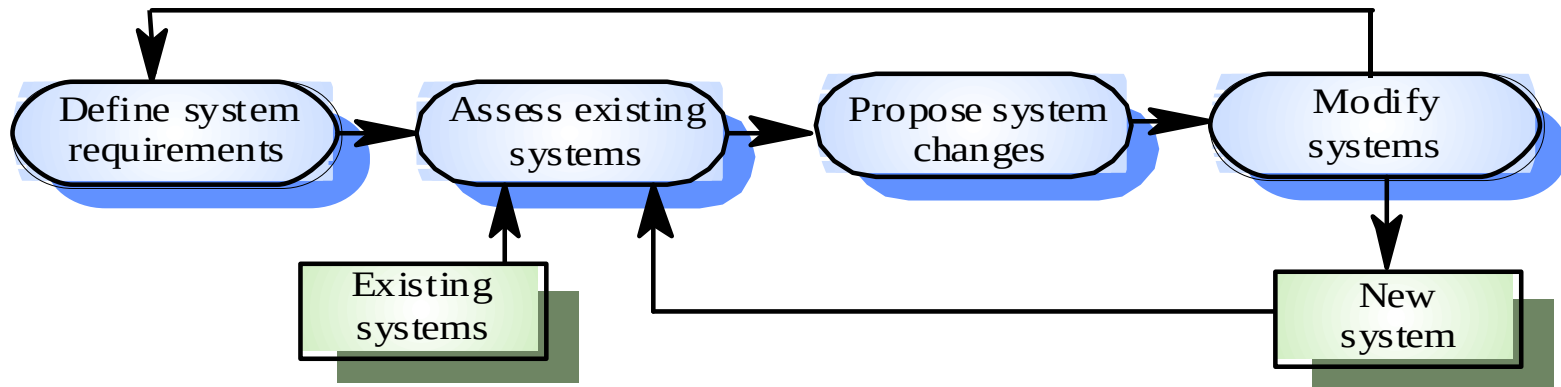
Testing phases



Software evolution

- Software is inherently **flexible** and can **change**.
- As requirements change through changing business circumstances, the software that supports the business must also evolve and change.
- Although there has been a demarcation between development and evolution (**maintenance**) this is increasingly irrelevant as fewer and fewer systems are completely new.

System Evolution



Software process models

- The goal of Software Engineering is to provide models and processes that lead to the production of well-documented maintainable software in a manner that is predictable.
- Also termed as software life cycle models
- A software process model is an abstract representation of a process.
 - Each process model represents a process from particular perspective.
- Examples of process perspectives are
 - Workflow perspective - sequence of activities
 - Data-flow perspective - information flow
 - Role/action perspective - who does what

Need for a software process model:

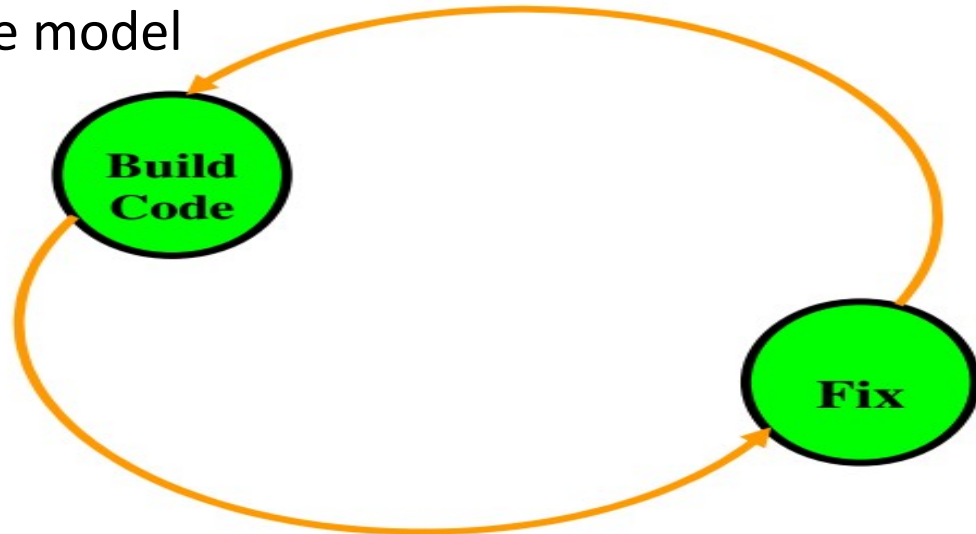
- The development team must identify a suitable software process model for the particular project and then adhere to it.
- Without using of a particular software model the development of a software product would not be in a systematic and disciplined manner.
- A software process model defines **entry** and **exit** criteria for every phase.
 - A phase can start only if its **phase-entry** criteria have been satisfied. So without software process model the entry and exit criteria for a phase cannot be recognized.
 - Without software process models it becomes difficult for software project managers to monitor the progress of the project.

Generic software process models

- Many process models have been proposed so far.
- Each of them has some advantages as well as some disadvantages.
- A few important and commonly used process models are:
 - Code-and-Fix process model
 - The waterfall process model
 - Spiral process Model
 - Incremental development process model
 - Evolutionary development process model
 - Formal Transformation process model
 - Reuse-based development process model
 - V-Model (Reading Assignment)
 - Time Boxing (Reading Assignment)
 - Extreme programming (Reading Assignment)

Code-and-Fix process model

- This model starts with an **informal** general product **idea** and just develops **code** until a product is "ready" (or **money** or **time** runs out). **Work is in random order.**
- **Product** is constructed **without** specifications or any **attempt** at design
- **Adhoc approach** and not well defined
- Simple two phase model



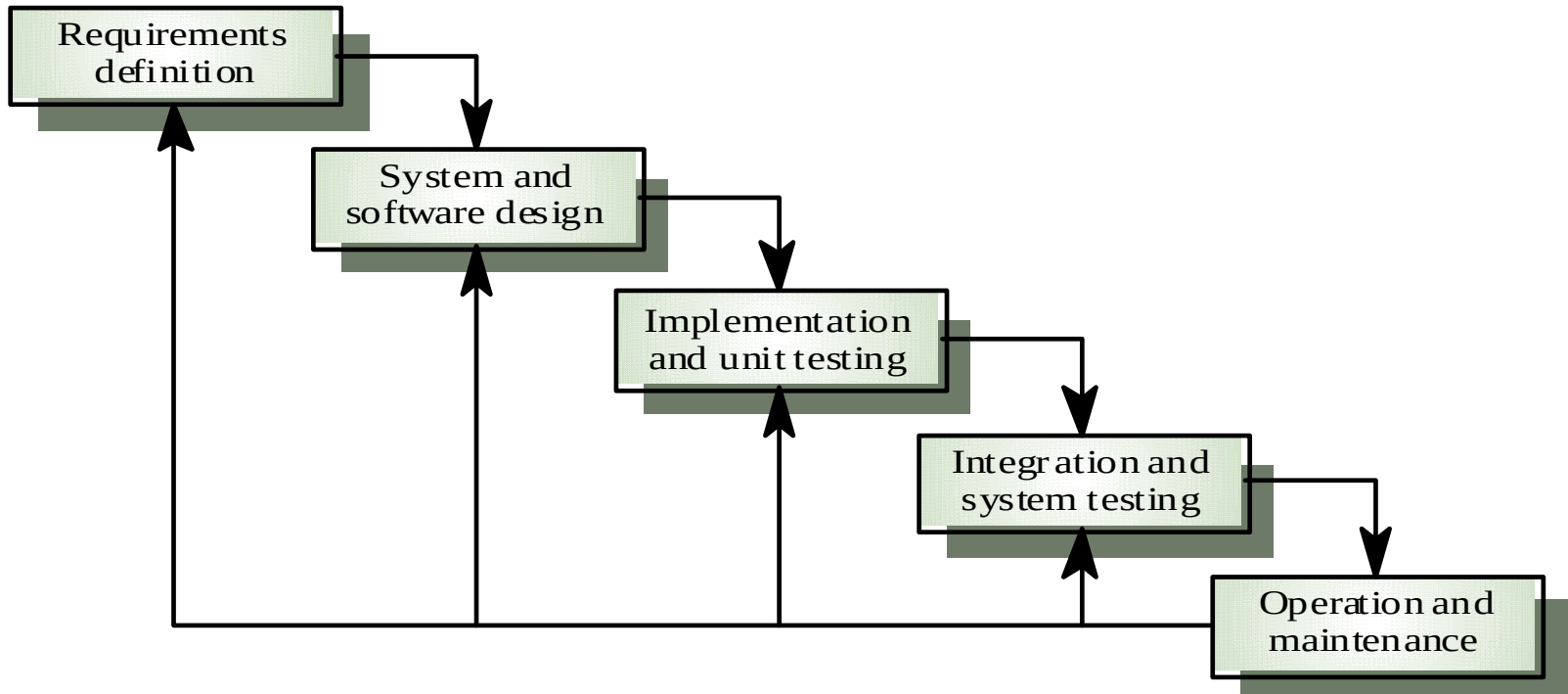
- Advantages of this model:
 - No administrative **overhead**
 - Signs of progress (**code**) early.
 - Low **expertise**, anyone can use it!
 - Useful for **small** projects.
 - Suitable for small **programming** exercises of 100 or 200 line

- disadvantages of this model:
 - Dangerous!
 - No visibility/control
 - No resource planning
 - No deadlines
 - Mistakes hard to detect/correct
 - Impossible for large projects
 - communication breakdown
 - Chaos/disorder.
 - No room for structured design
 - Maintenance is practically not possible
 - Code soon becomes unfixable & unenhanceable

Waterfall model

- Is also called **linear sequential** model
- Waterfall model is intuitively the **most** obvious way to develop software. This model can be considered to be a **theoretical way of developing software**. But all other process models are essentially derived from the waterfall model
- The following phase should **not** start until the previous phase has finished: no **feedback**
- But in practice, phases overlap and may return to a previous phase.
- The phases partition the project, each addressing a separate concern

Waterfall model cont...



Cont...

- This model is **easy to understand** and **reinforces** the notion of “**define before design**” and “**design before code**”.
- The model expects **complete** & **accurate** requirements early in the process, which is **unrealistic**
- There are separate and distinct identified phases of specification and development in the waterfall model:
 - Requirements analysis and definition
 - System and software design
 - Implementation and unit testing
 - Integration and system testing
 - Operation and maintenance

Cont...

- Advantages of waterfall model:
 - Easy to understand and implement.
 - Widely used and known (in theory!)
 - Fits other engineering process models: civil, mechanical etc.
 - Reinforces good habits: define-before- design, design-before-code
 - Identifies deliverables and milestones
 - Works well on large/mature products and weak teams.

Cont...

- The drawback of the waterfall model:
 - Requires that requirements are **well-specified** and **unchanging**.
 - It is difficult to define all **requirements** at the beginning of a project
 - Not suitable for **accommodating** any **change/expandability problem**
 - Difficulty of accommodating change after the process is underway
 - Error of any phase is detected at the **end** of the entire lifecycle
 - It does not **allow** for very much iteration.
 - Real projects are **rarely sequential**

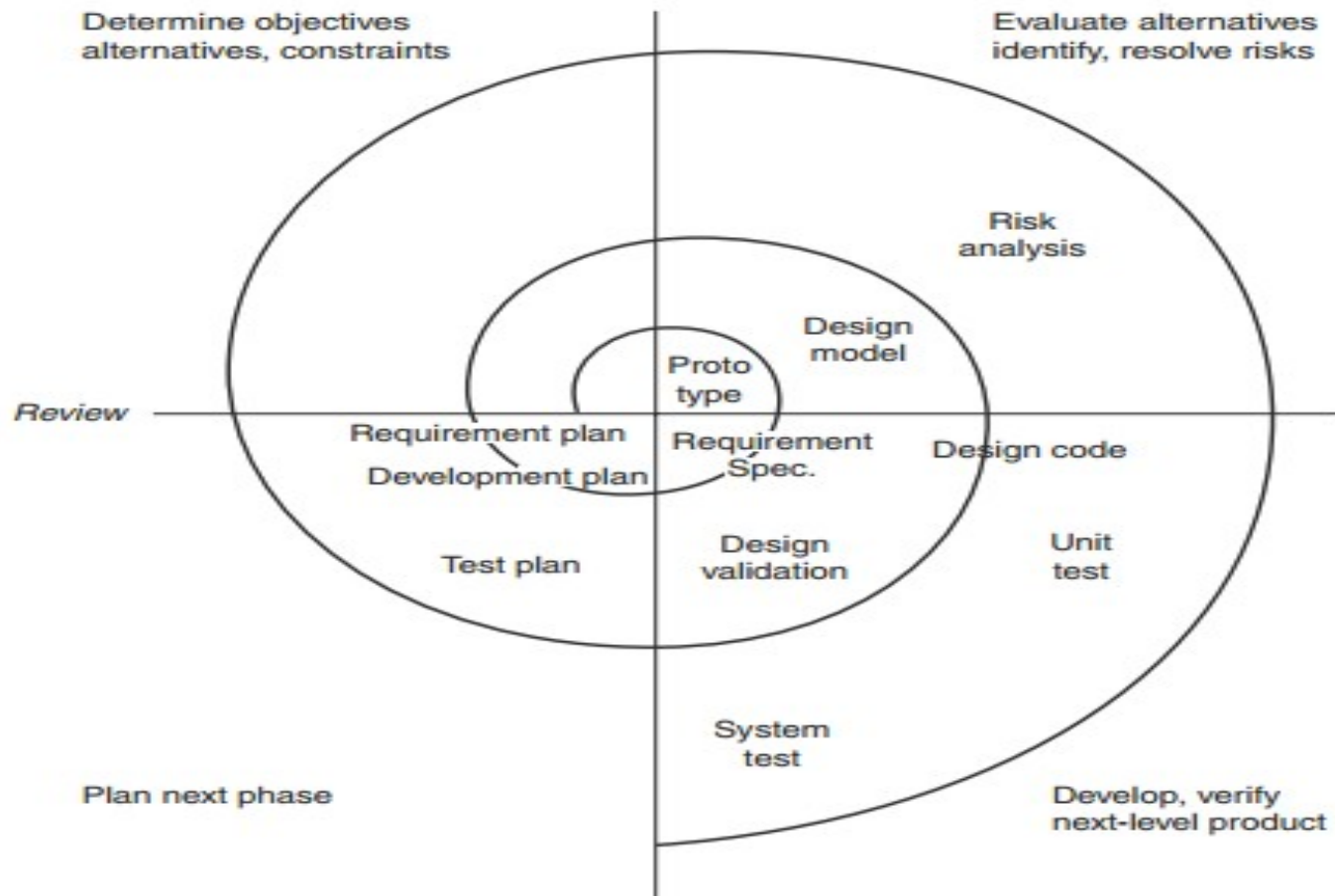
Cont...

- Working version becomes available very late (after coding)
 - It does not scale up well to large projects.
 - Developers may have to wait due to dependent tasks
- This model is only appropriate when the requirements are well-understood, short duration projects and automation of existing manual systems

Spiral process Model

- Process is represented as a **spiral** rather than as a **sequence** of activities with backtracking.
- Extends waterfall model by adding iteration to explore/manage risk.
- A spiral model is **divided** into a number of **framework** activities, also called ***task regions***.
- Typically, there are between **three** and **six** task regions.
- **Delivers** the software in a series of **incremental** releases
- When risks are **high** and **need** to be **resolved**, it is reasonable to use this approach

Cont...



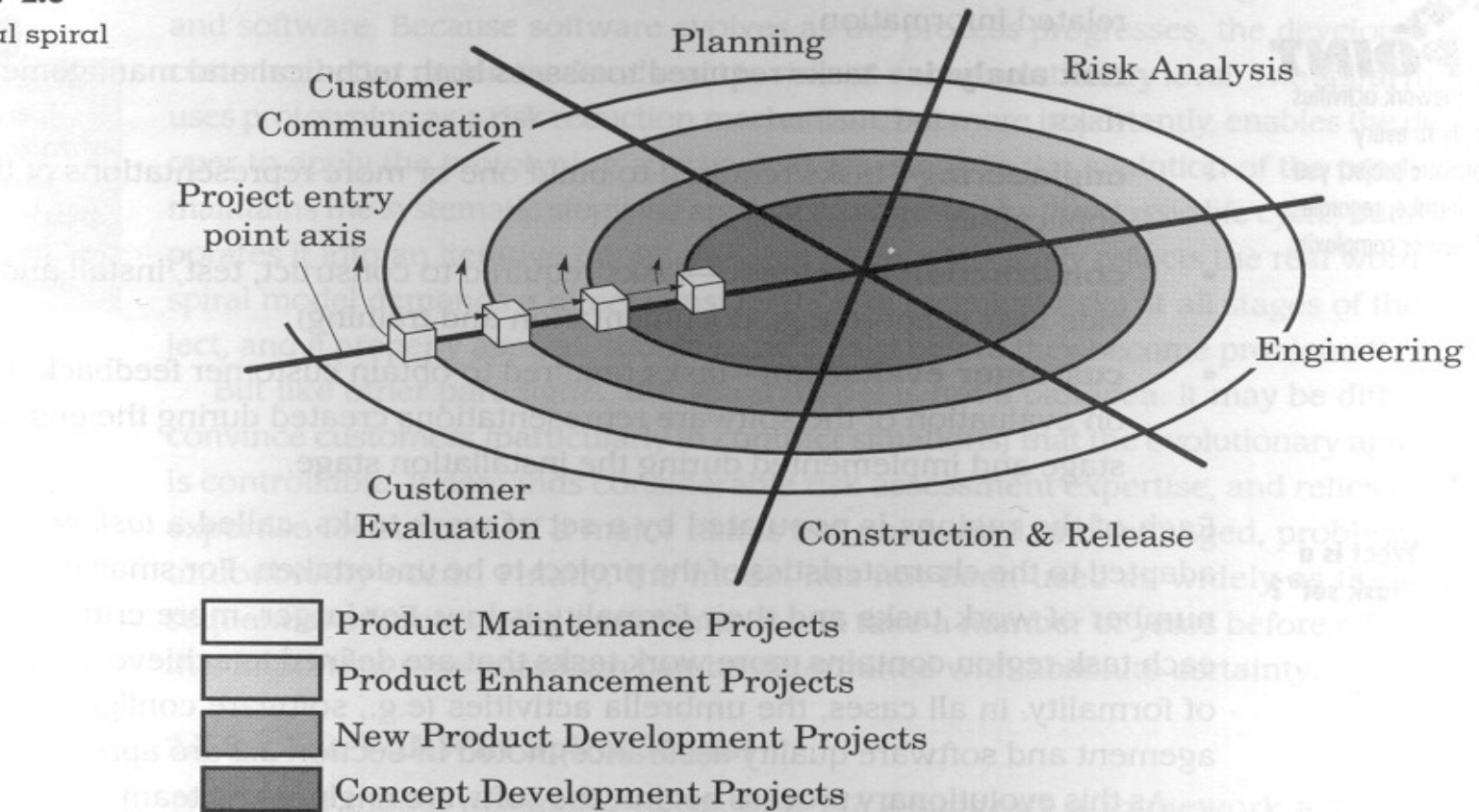
Cont...

- Objective setting
 - Specific objectives for the phase are identified
- Risk assessment and reduction
 - Risks are assessed and activities put in place to reduce the key risks
- Development and validation
 - A development model for the system is chosen which can be any of the generic models
- Planning
 - The project is reviewed and the next phase of the spiral is planned

Cont...

FIGURE 2.8

A typical spiral model



Cont...

- *customer communication* – tasks required to establish effective communication between **developer** and **customer**.
- *planning* – tasks required to define **resources**, timelines and other project related information
- *risk analysis* – tasks required to assess both **technical and management risks**
- *engineering* – tasks required to build one or more representations of the application
- *construction and release* – tasks required to construct, test, install and provide user support (e.g. documentation & training)
- *customer evaluation* – tasks required to get customer feedback on evaluation of the software representations created during the engineering stage and implemented during the installation stage

Cont...

- Advantages:
 - *Realism*: the model **accurately** reflects the iterative nature of software development on projects with **unclear** requirements.
 - *Flexible*: incorporates the advantages of the **waterfall** and **evolutionary** methods.
 - Comprehensive model to decreases risk.
 - Good project **visibility**.
 - On each iteration, plans, costs, risks and schedules updated and project manager get more accurate estimate of number of required iterations

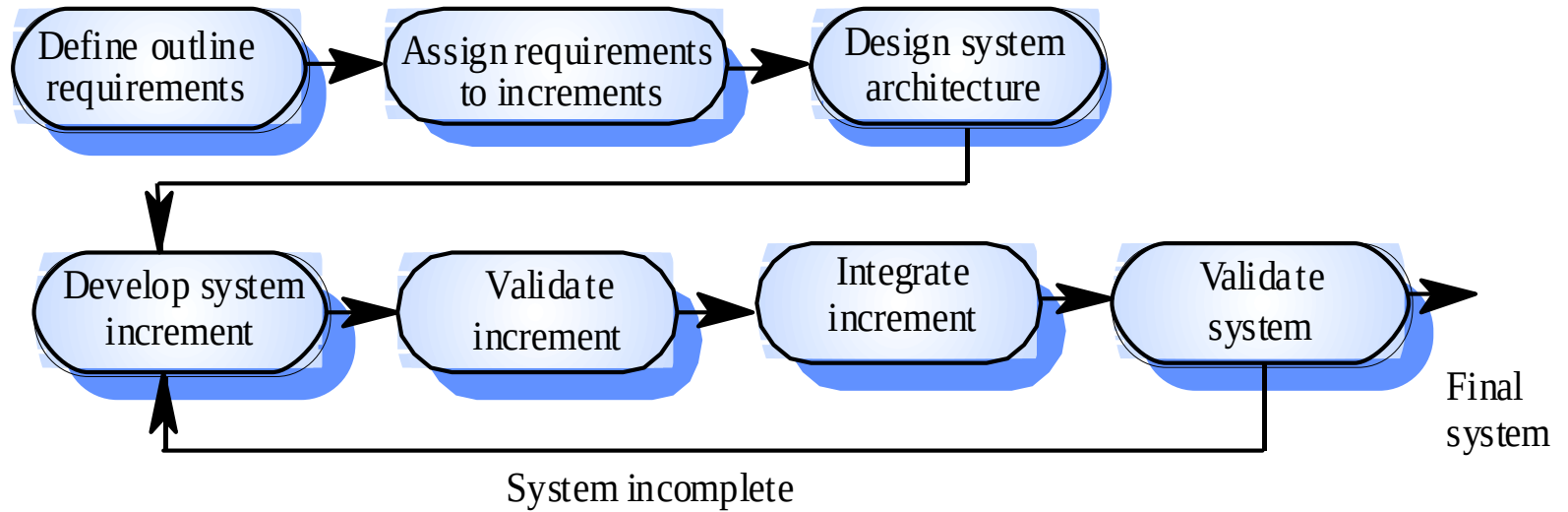
Cont...

- Spiral Process Model Problems
 - Needs technical expertise in risk analysis and risk management to work well.
 - Model is poorly understood by non technical management, hence not so widely used.
 - Complicated model, needs competent professional management.
 - High administrative overhead.

Incremental development model

- Rather than deliver the system as a **single delivery**,
 - **Development** and **delivery** is broken down into **increments**;
 - With each **increment** delivering part of the required functionality.
- **User requirements are prioritised and the highest priority requirements are included in early increments.**
- Once the development of an increment is started, the requirements are frozen though requirements for later increments can continue to evolve.

Cont...



Cont...

- Incremental development model advantages:
 - Customer value can be delivered with each increment so system **functionality is available earlier.**
 - Client does not have to pay for entire software together.
 - Early increments act as a prototype to help elicit requirements for later increments
 - Lower risk of overall project failure.
 - The highest priority system services tend to receive the most testing.

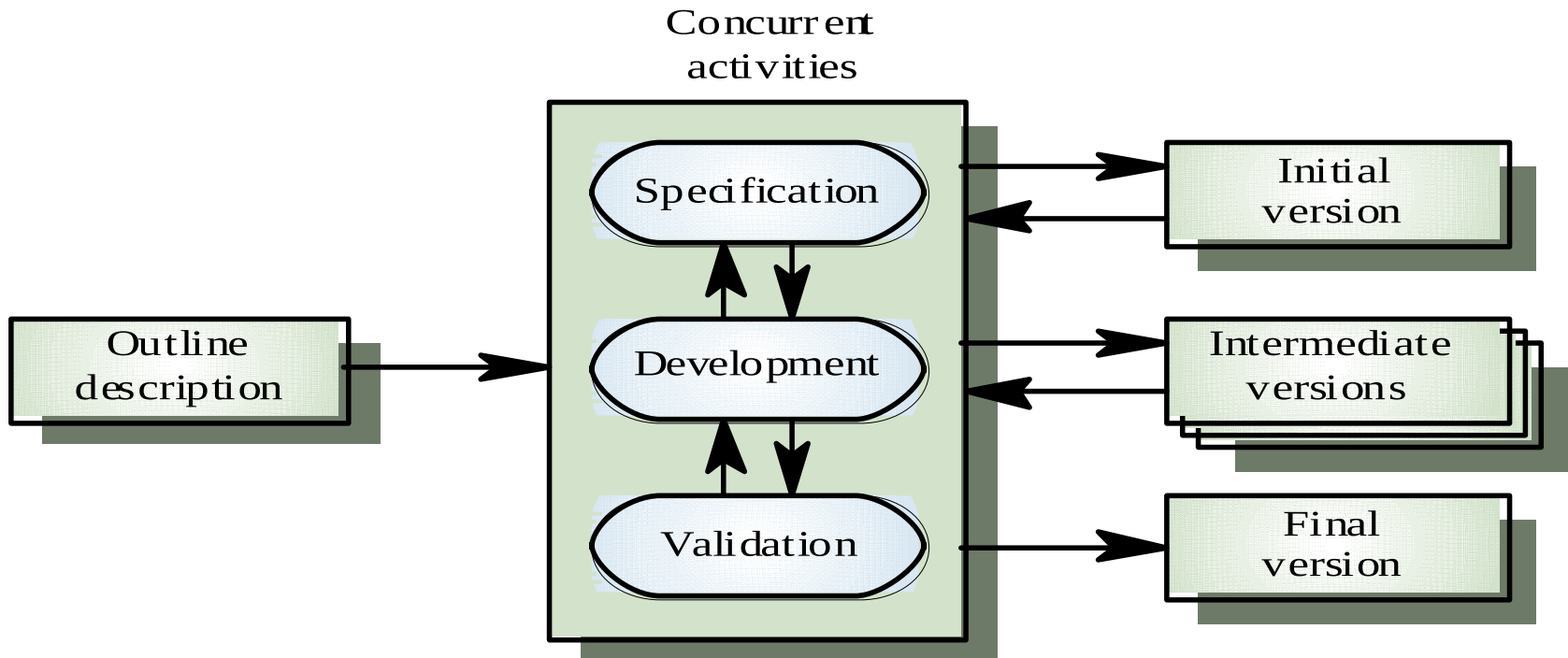
Cont...

- Incremental development model problems:
 - Increments may be difficult to define
 - Software may be difficult to maintain
- When the “core” product is well understood and increments can be easily defined, it is reasonable to use this approach.

Evolutionary development model

- Evolutionary development is based on the idea of developing an **initially** implementation,
 - exposing this to **user comment** and
 - **refining** this through many versions until an **adequate** system development.
- When requirements are **not** well **understood**, it may be reasonable to use this approach.
- A prototype's **purpose** is to **understand** requirements. A better (**good quality**) product must be **subsequently** developed.

Cont...



Cont...

- There are two types of evolutionary development
- Exploratory development
 - Objective is to work with **customers** and to **evolve** a final system from an **initial outline specification**.
 - Should start with **well-understood** requirements.
 - The system evolves by **adding new features** as they are proposed by customer.

Cont...

- Throw-away prototyping
 - Objective is to understand the system requirements.
Should start with poorly understood requirements
 - Develop “quick and dirty” system quickly;
 - Expose to user comment;
 - Refine;
 - Until adequate system is developed.
 - Particularly suitable where:
 - detailed requirements not possible;
 - powerful development tools (e.g. GUI) available

Cont...

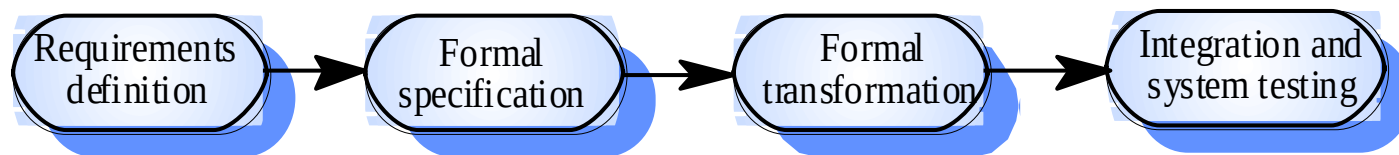
- Evolutionary development model advantages:
 - The resulting system is easier to use
 - User needs are better accommodated
 - Problems are detected earlier
 - The design is of higher quality
 - The development incurs less effort
- Evolutionary development model Problems:
 - Lack of process visibility
 - Systems are often poorly structured
 - Special skills (e.g. in languages for rapid prototyping) may be required

Cont...

- The users and the designers must be well aware of the issues and the pitfalls
- Use **prototyping** when the requirements are unclear.
- Applicability
 - For small or medium-size interactive systems
 - For parts of large systems (e.g. the user interface)
 - For short-lifetime systems

Formal transformation development process

- Based on the transformation of a mathematical specification through different representations to an executable program
- Transformations are '**correctness-preserving**' so it is straightforward to show that the program conforms to its specification
- Embodied in the 'Cleanroom' approach to software development



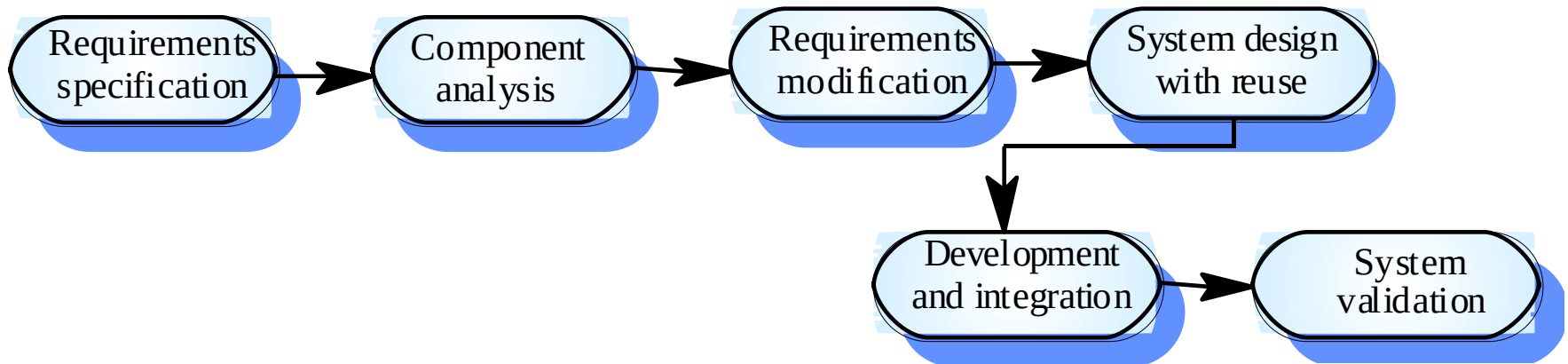
Cont...

- Problems
 - Need for specialised skills and training to apply the technique
 - Difficult to formally specify some aspects of the system such as the user interface.
- Advantage
 - The resulting software will have high quality
- Applicability
 - Critical systems especially those where a safety or security case must be made before the system is put into operation

Reuse-oriented development

- Based on **systematic reuse where** systems are integrated **from existing components** or **COTS** (Commercial-off-the-shelf) systems
- Process stages
 - Component analysis
 - Requirements modification
 - System design with reuse
 - Development and integration
- **This approach is becoming more important but still limited experience with it**

Cont...



- Software systems are **intangible** so managers need documents to **assess** progress.

Process Models & Process visibility

Process model	Process visibility
Waterfall model	Good visibility, each activity produces some deliverable
Evolutionary development	Poor visibility, uneconomic to produce documents during rapid iteration
Formal transformations	Good visibility, documents must be produced from each phase for the process to continue
Reuse-oriented development	Moderate visibility, it may be artificial to produce documents describing reuse and reusable components.
Spiral model	Good visibility, each segment and each ring of the spiral should produce some document.

Automated process support (CASE)

- Computer-aided software engineering (CASE) is software to support software development and evolution processes
- Activity automation
 - Graphical editors for system model development
 - Data dictionary to manage design entities
 - Graphical UI builder for user interface construction
 - Debuggers to support program fault finding
 - Automated translators to generate new versions of a program

Case technology

- Case technology has led to significant improvements in the software process though not the order of magnitude improvements that were once predicted
 - Software engineering requires creative thought - this is not readily automatable
 - Software engineering is a team activity and, for large projects, much time is spent in team interactions. CASE technology does not really support these

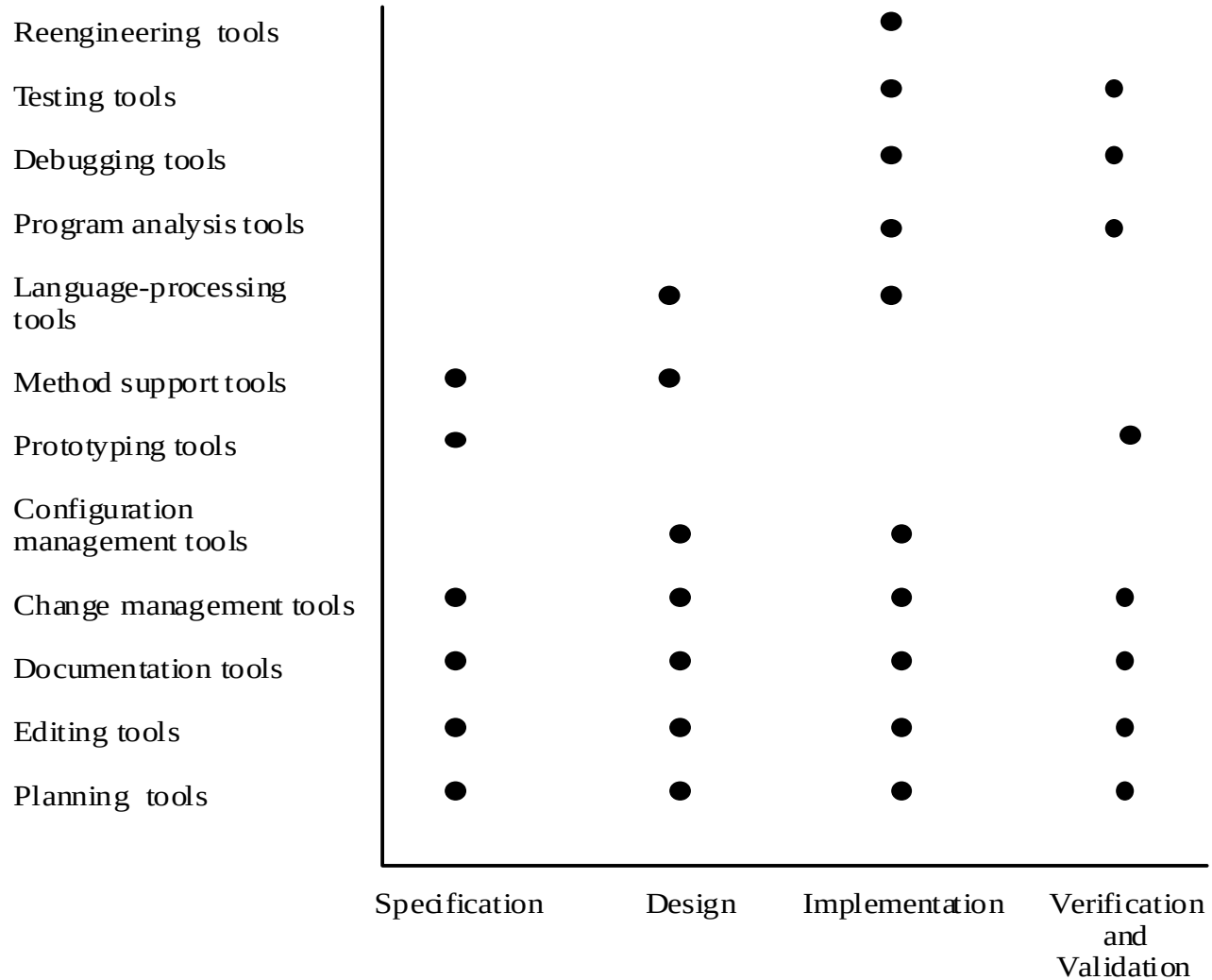
CASE classification

- Classification helps us understand the different types of CASE tools and their support for process activities
- Functional perspective
 - Tools are classified according to their **specific function**
- Process perspective
 - Tools are classified according to **process** activities that are supported
- Integration perspective
 - Tools are classified according to their **organisation** into **integrated** units

Functional tool classification

Tool type	Examples
Planning tools	PERT tools, estimation tools, spreadsheets
Editing tools	Text editors, diagram editors, word processors
Change management tools	Requirements traceability tools, change control systems
Configuration management tools	Version management systems, system building tools
Prototyping tools	Very high-level languages, user interface generators
Method-support tools	Design editors, data dictionaries, code generators
Language-processing tools	Compilers, interpreters
Program analysis tools	Cross reference generators, static analysers, dynamic analysers
Testing tools	Test data generators, file comparators
Debugging tools	Interactive debugging systems
Documentation tools	Page layout programs, image editors
Re-engineering tools	Cross-reference systems, program re-structuring systems

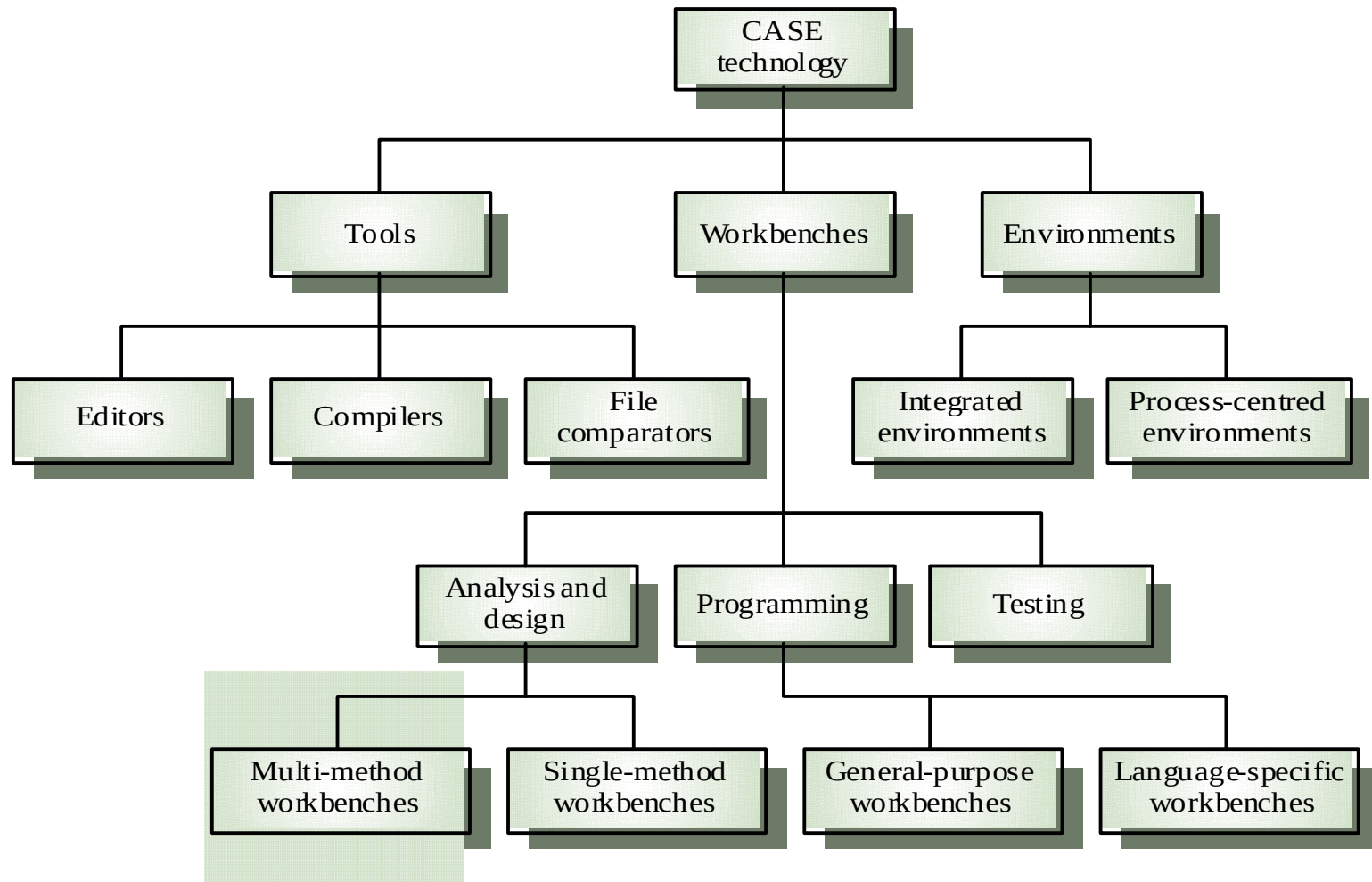
Activity-based classification



CASE integration

- Tools
 - Support individual process tasks such as design consistency checking, text editing, etc.
- Workbenches
 - Support a process phase such as **specification** or **design**, Normally include a **number** of **integrated** tools
- Environments
 - Support all or a **substantial** part of an **entire** software process. Normally include several **integrated workbenches**

Tools, workbenches, environments



Summary

- Software processes are the activities involved in producing and evolving a software system. They are represented in a software process model
- General activities are specification, design and implementation, validation and evolution
- Requirements engineering is the process of developing a software **specification**
- Design and implementation processes transform the specification to an **executable** program
- Validation involves **checking** that the system meets to its specification and user needs

Summary cont...

- **Evolution** is concerned with **modifying** the system after it is in use
- Generic process models describe the **organisation** of software processes
- **Iterative** process models describe the software process as a **cycle** of activities
- CASE **technology** supports software **process activities**